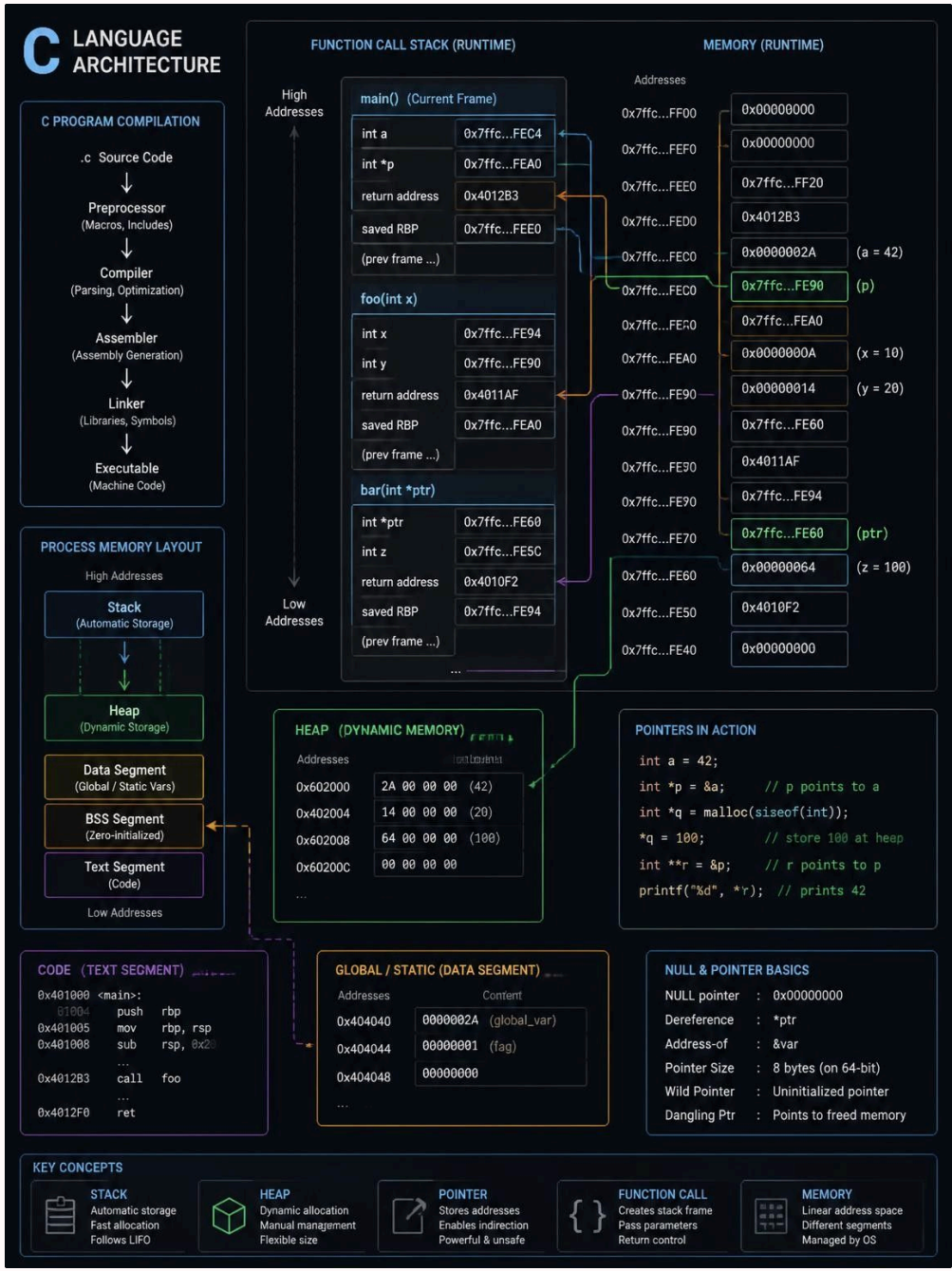
Puntatori e Funzioni in C

Parametri per riferimento, funzioni che restituiscono puntatori, puntatori a funzione

LINGUAGGIO C | PUNTATORI | FUNZIONI



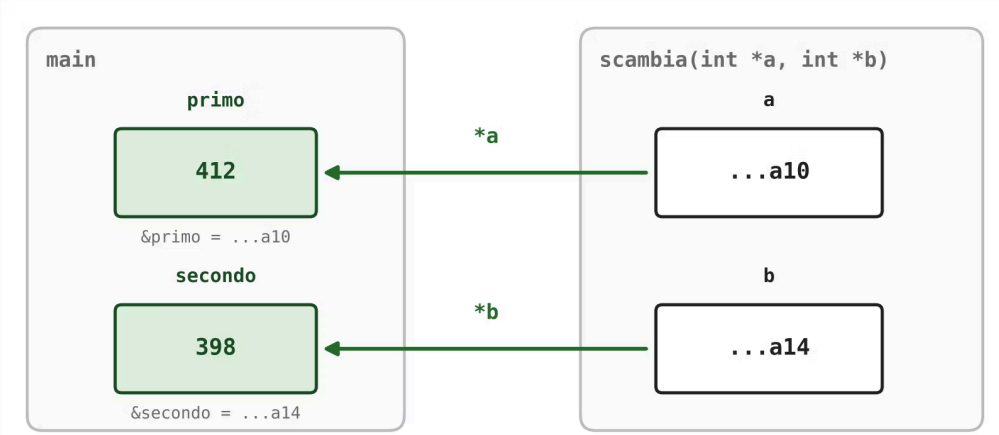
Copie e Indirizzi

Il problema: una funzione riceve copie dei suoi argomenti, quindi non può modificare le variabili del chiamante.

La soluzione: si passa l'indirizzo della variabile; nella funzione si scrive nella variabile puntata con *a.

```
void scambia(int *a, int *b)
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

scambia(&primo, &secondo);
```



Chiamata: scambia(&primo, &secondo). a e b contengono gli indirizzi (esadecimali, abbreviati) delle variabili del main: scrivendo *a e *b la funzione modifica primo e secondo.

Una funzione che ha già ricevuto un puntatore lo passa ad altre funzioni senza &: dentro ordina_tre si scrive scambia(a, b), perché a e b sono già indirizzi.

Tre Usi di un Parametro Puntatore

Uso	Esempio	La funzione
uscita	scomponi(8130, &h, &m, &s)	scrive un risultato
ingresso e uscita	preleva(&saldo, 550)	legge il valore e lo aggiorna
uscita facoltativa	calcola(..., &totale, NULL, NULL)	scrive solo se il puntatore non è NULL

return restituisce un solo valore: quando i risultati sono più di uno, escono dai parametri puntatore.

```
void scomponi(int totale, int *ore, int *minuti, int
*secondi)
{
   *ore = totale / 3600;
   *minuti = totale % 3600 / 60;
   *secondi = totale % 60;
}
```

return per l'Esito, Puntatori per i Risultati

Il problema: il prelievo dalla tessera della mensa può non riuscire, e il chiamante deve saperlo.

La soluzione: la funzione restituisce 1 se l'operazione è riuscita, 0 altrimenti; la variabile del chiamante si modifica solo in caso di successo.

Esempi: resto del distributore (esito, monete usate, importo mancante), lettura di una data (1 valida, 0 non valida, -1 fine), volo del drone rifiutato sotto la riserva.

```
int preleva(int *saldo, int importo)
{
    int riuscito = 0;

    if (*saldo >= importo) {
        *saldo = *saldo - importo;
        riuscito = 1;
    }
    return riuscito;
}

if (preleva(&saldo, PASTO)) { ... }
```

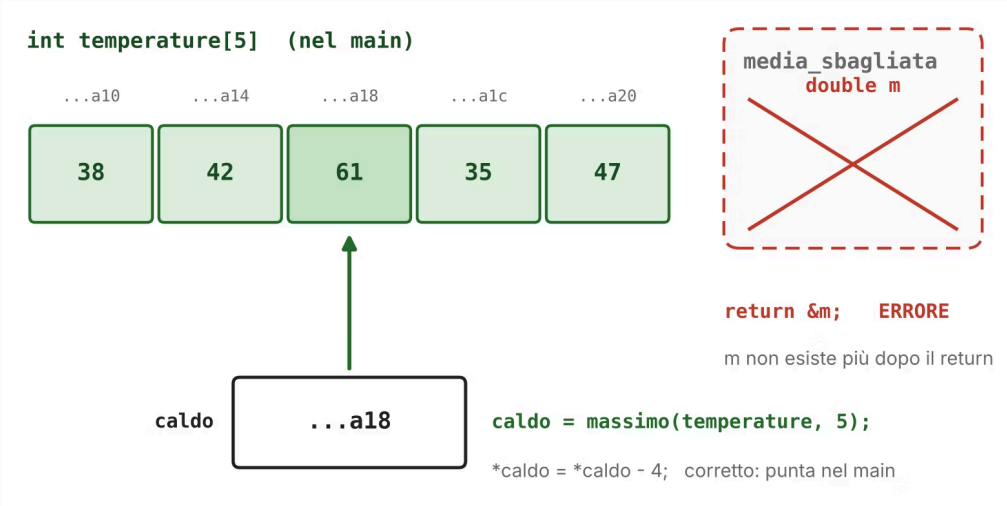
Una Funzione che Restituisce un Puntatore

Il problema: trovare il frigorifero più caldo della mensa e abbassarne la temperatura.

La soluzione: la ricerca restituisce il puntatore all'elemento trovato; il chiamante lo legge e lo modifica direttamente.

```
int *massimo(int *v, int n);

int *caldo = massimo(temperature, N);
*caldo = *caldo - 4;
posizione = caldo - temperature;
```



Se il chiamante deve solo leggere, parametro e risultato sono const: const int *minimo(const int *v, int n).

NULL per "Non Trovato"

Una ricerca che restituisce un puntatore dice "non trovato" restituendo NULL. Il chiamante controlla sempre il risultato prima di usarlo.

Le versioni a mano di strstr (trova) e strrchr (ultimo) restituiscono un puntatore dentro la stringa ricevuta, oppure NULL.

```
int *cerca(int *v, int n, int valore)
{
    int *p = v;

    while (p < v + n && *p != valore) {
        p++;
    }
    return p < v + n ? p : NULL;
}

int *trovato = cerca(armadietti, N, matricola);
if (trovato != NULL) {
    *trovato = 0;       /* armadietto liberato */
}
```

Quale Indirizzo si Può Restituire

Indirizzo restituito	Valido dopo il return?
un elemento di un array del chiamante	sì
una parte di una stringa ricevuta (testo + k)	sì
NULL	sì, ma non si dereferenzia
una variabile locale o un parametro	no: la variabile non esiste più

Al posto dell'indirizzo di una variabile locale si restituisce il valore, oppure si scrive il risultato in un parametro di uscita.

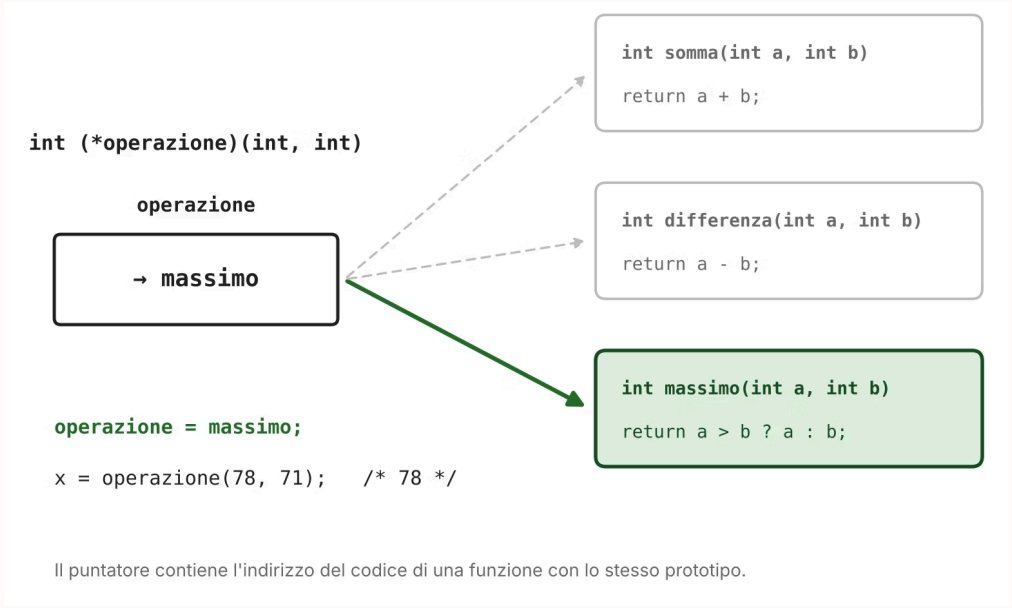
```
double *media_sbagliata(const int *voti, int n)
{
   double m;
   ...
   return &m;       /* ERRORE */
}
```

Il Puntatore a Funzione

Anche il codice di una funzione ha un indirizzo. Un puntatore a funzione lo contiene e permette di chiamare la funzione scelta in quel momento.

Scrittura	Significato
int (*f)(int, int)	puntatore a una funzione (int, int) che restituisce int
int *f(int, int)	funzione che restituisce int * (senza parentesi)
f = somma	f punta a somma
f(a, b)	chiamata tramite il puntatore

Il puntatore può ricevere solo funzioni con lo stesso prototipo, e si confronta con == (operazione == dividi, operazione == NULL).



Il puntatore contiene l'indirizzo del codice di una funzione con lo stesso prototipo.

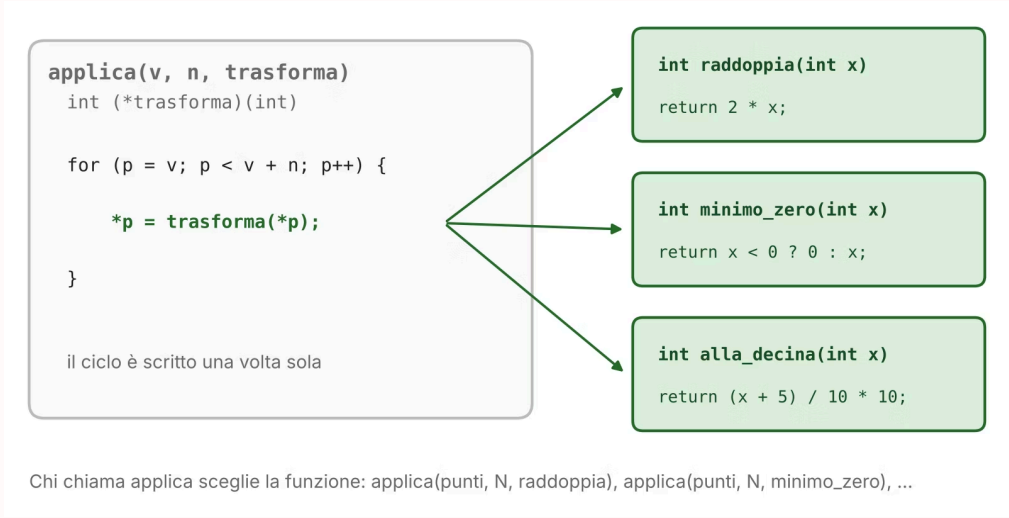
La Funzione come Parametro

Il problema: applicare trasformazioni diverse ai punteggi di una gara senza riscrivere ogni volta il ciclo.

La soluzione: una funzione generale riceve, oltre ai dati, la funzione da applicare (callback).

```
void applica(int *v, int n, int (*trasforma)(int))
{
    int *p;
    for (p = v; p < v + n; p++) {
        *p = trasforma(*p);
    }
}
applica(punti, N, raddoppia);
```

Si passa il nome della funzione, senza (): applica(punti, N, raddoppia), non raddoppia(3).



Chi chiama applica sceglie la funzione: applica(punti, N, raddoppia), applica(punti, N, minimo_zero), ...

Criteri, Condizioni e Formule

Lo stesso schema risolve molti problemi: il ciclo è scritto una volta, ciò che cambia arriva come funzione.

Funzione generale	Funzione passata	Esempio
migliore(v, n, preferisci)	int (int, int)	consegna più rapida, più lenta, più vicina ai 30 minuti
conta_se(v, n, condizione)	int (int)	misure di rumore oltre 80 dB
converti(in, out, n, formula)	double (double)	Celsius in Fahrenheit e kelvin
trasforma(testo, regola)	char (char)	maiuscolo, Cesare, cifre oscurate
vincitore(punti, n, valuta)	int (const int *)	triathlon: totale, prova migliore, prova peggiore

```
const int *migliore(const int *v, int n, int
(*preferisci)(int, int))
{
    const int *p;
    const int *scelto = v;

    for (p = v + 1; p < v + n; p++) {
        if (preferisci(*p, *scelto)) {
            scelto = p;
        }
    }
    return scelto;
}
```

Scegliere la Funzione a Tempo di Esecuzione

Il problema: la calcolatrice legge l'operatore e deve eseguire l'operazione corrispondente.

La soluzione: una funzione restituisce il puntatore alla funzione di calcolo, NULL se l'operatore non esiste.

La dichiarazione si legge dall'interno: scegli_operazione è una funzione (char) che restituisce un puntatore a una funzione (double, double) che restituisce double.

```
double (*scegli_operazione(char simbolo))(double, double);


operazione = scegli_operazione(simbolo);
if (operazione == NULL) {
    printf("operatore sconosciuto\n");
} else if (operazione == dividi && b == 0) {
    printf("divisione per zero\n");
} else {
    printf("%g\n", operazione(a, b));
}
```

Errori Comuni da Evitare

1

Passare il valore invece dell'indirizzo: scambia(primo, secondo).

2

Aggiungere & a un puntatore già ricevuto: scambia(&a, &b) dentro ordina_tre.

3

Scrivere x = 0 invece di *x = 0 nella funzione.

4

Modificare le uscite anche quando l'operazione fallisce.

5

Restituire l'indirizzo di una variabile locale.

6

Usare il puntatore restituito senza controllare NULL.

7

Dichiarare int *f(int, int) invece di int (*f)(int, int).

8

Passare raddoppia(3) invece di raddoppia a una funzione che vuole una callback.

Riepilogo Concetti

Operazione	Tecnica	Note
Modificare una variabile del chiamante	void f(int *x), f(&v)	nella funzione *x
Più risultati	parametri di uscita	return libero per l'esito
Esito dell'operazione	return 1 / 0	uscite scritte solo se riesce
Risultati facoltativi	parametro NULL	controllo prima di scrivere
Restituire un elemento	int *f(int *v, int n)	punta nell'array del chiamante
Non trovato	return NULL	controllo del chiamante
Sola lettura	const int *f(const int *v, ...)	risultato const
Puntatore a funzione	int (*f)(int, int)	stesso prototipo
Callback	f(v, n, funzione)	nome senza ()
Scelta della funzione	funzione che restituisce un puntatore a funzione	NULL se non prevista

SEZIONE 1

Esempi Pratici: Parametri per Riferimento

Concetti base — tre file su scambio, ingresso e uscita, più risultati.

1 – 010-scambio-e-ordine.c

Problema: tempi dei primi tre di una corsa campestre.

Tecnica: copie e indirizzi, scambia, ordina_tre senza &.

2 – 020-parametri-in-out.c

Problema: tessera prepagata della mensa.

Tecnica: saldo in ingresso e uscita, return 1/0.

3 – 030-piu-risultati.c

Problema: durata delle registrazioni di una videocamera.

Tecnica: secondi in ore, minuti e secondi; giorni con return.

Esempi Pratici: Problemi con Parametri per Riferimento

040 – Resto in monete

Distributore con scorta limitata: esito, monete usate e importo mancante.

050 – Durata dell'allenamento

Tempi dei giri sommati con il riporto, giro migliore in ingresso e uscita.

060 – Lettura di una data

Prenotazione della palestra: esito a tre valori e giorno successivo.

070 – Batteria del drone

Carica e minuti di volo aggiornati solo se la manovra è possibile.

080 – Scontrino con sconto

Totale, sconto e punti fedeltà come uscite facoltative.

SEZIONE 2

Esempi Pratici: Funzioni che Restituiscono Puntatori

Concetti base — tre file sul puntatore come valore di ritorno.

1 – 010-puntatore-al-massimo.c

Problema: frigoriferi della mensa troppo caldi.

Tecnica: int *massimo per modificare, const int *minimo per leggere.

2 – 020-ricerca-con-null.c

Problema: armadietti degli studenti.

Tecnica: ricerca che restituisce NULL se non trova.

3 – 030-mai-indirizzi-locali.c

Problema: media dei voti di uno studente.

Tecnica: perché non si restituisce &locale, tre alternative corrette.

Esempi Pratici: Problemi con Puntatori Restituiti

040 – Scaffale del magazzino

Puntatore alla quantità di un componente cercato per codice.

050 – Prossimo treno

Puntatore al prossimo orario e treni successivi p + 1, p + 2.

060 – Cerca parola

Ricerca nel regolamento con una strstr scritta a mano.

070 – Nome del file

Cartella, nome ed estensione con una strrchr scritta a mano.

080 – Aula libera

Puntatore all'aula libera più vicina a quella richiesta.

SEZIONE 3

Esempi Pratici: Puntatori a Funzione

Concetti base — tre file su dichiarazione, callback e criteri.

1 – 010-dichiarazione-e-chiamata.c

Problema: punteggio di una partita di basket.

Tecnica: int (*operazione)(int, int), assegnazione e chiamata.

2 – 020-funzione-come-parametro.c

Problema: punteggi di una gara di matematica.

Tecnica: applica(v, n, trasforma) con tre trasformazioni.

3 – 030-criterio-di-scelta.c

Problema: tempi di consegna di una pizzeria.

Tecnica: migliore(v, n, preferisci) con tre criteri.

SEZIONE 3

Esempi Pratici: Problemi con Puntatori a Funzione

040 – Calcolatrice

L'operatore letto sceglie la funzione di calcolo.

050 – Filtro delle misure

Rumore in mensa: conta_se ed elenca_se con tre condizioni.

060 – Conversioni

Stazione meteo: Celsius in Fahrenheit e kelvin con una sola funzione.

070 – Trasforma testo

Maiuscolo, cifrario di Cesare e cifre oscurate, carattere per carattere.

080 – Classifica con criterio

Triathlon scolastico: tre premi con tre criteri diversi.